

Radio

Live channel messages meet relational queries.



Install & load

```
INSTALL radio FROM community;
LOAD radio;
```

01 At a glance

Subscribe to WebSocket or Redis Pub/Sub endpoints, query buffered messages, and queue replies from DuckDB. The subscription URL identifies the feed throughout its lifecycle, while background I/O keeps network activity separate from buffer scans.

Live channel → Memory buffer → DuckDB rows

02 Subscription lifecycle

`CALL radio_subscribe(url)`

Open a feed; `receive_message_capacity` tunes its buffer size.

`radio_subscriptions()`

Inspect active feeds and transmission success or failure counters.

`radio_received_messages()`

Read buffered messages across all subscriptions.

`radio_subscription_received_messages(url)`

Read the buffer for one subscription URL.

`CALL radio_listen(first, wait)`

Wait for arrivals, bounded by an `INTERVAL` timeout.

`CALL radio_transmit_message(...)`

Queue URL, channel, BLOB payload, attempt count and retry interval.

`CALL radio_unsubscribe(url)`

Close a feed using the exact subscription URL.

03 Keep in mind

A buffer is not durable storage

Subscriptions and messages live in process memory. Capacity limits can evict older messages; process exit discards state. Persist records you need in a DuckDB table and define deduplication around `subscription_id` and `message_id`.

04 Subscribe, wait and inspect

REQUIRES SETUP

```
SET VARIABLE feed = 'ws://localhost:8765/';
CALL radio_subscribe(getvariable('feed'));
CALL radio_listen(true, INTERVAL '5 seconds');
SELECT receive_time, channel,
       decode(message) AS text
FROM radio_received_messages()
WHERE subscription_url = getvariable('feed')
ORDER BY receive_time DESC
LIMIT 20;
```

Requires a WebSocket endpoint on `localhost:8765` that emits UTF-8 text messages.

05 Queue a reply and inspect delivery

REQUIRES SETUP

```
CALL radio_transmit_message(
  getvariable('feed'), 'main',
  '{"type": "ack"}'::BLOB,
  3, INTERVAL '500 milliseconds'
);
SELECT transmit_successes, transmit_failures
FROM radio_subscriptions();
```

Poll until delivery completes before calling `radio_unsubscribe`. Your server defines channel semantics.

06 Check the extension build

```
SELECT radio_version();
```

Local diagnostic; no subscription is opened.

07 In practice

Payloads and delivery

`message` is a BLOB. Use `decode(message)` for UTF-8 text, then JSON functions if appropriate. `Transmit` calls queue work; check subscription counters for delivery progress. A queued message does not imply successful delivery.

[Full docs & examples ↗](#) [Source ↗](#)